

Dominator Analysis

Obiettivo della dominator analysis

Determinare quali nodi devono essere necessariamente attraversati per arrivare ad un altro nodo

• Utilizzi

È alla base dell'ottimizzazione dei compilatori, utilizzata per individuare quali parti di un programma vengono eseguite necessariamente prima di altre all'interno di un CFG (**Control Flow Graph**).

Questa informazione è fondamentale per costruire strutture come il dominator tree, che rappresenta la gerarchia del flusso di controllo.

Grazie ai **dominatori** (che sono essenzialmente nodi che il programma dovrà necessariamente attraversare per arrivare ad un altro nodo) si possono applicare ottimizzazioni sicure, come lo spostamento di istruzioni in punti che dominano tutti i loro usi o l'eliminazione di **codice inutile** (**Dead Code**).

Inoltre, è essenziale per costruire la **SSA (Static Single Assignment)**, inserendo correttamente le funzioni **phi**.

In generale, aiuta a garantire che le trasformazioni del codice preservino il comportamento del programma.

• Quando un nodo domina un altro nodo?

In un CFG, un nodo X **domina** un altro nodo Y **se e solo se** il nodo X compare in ogni possibile percorso del CFG dal nodo ENTRY al nodo Y.

Es:

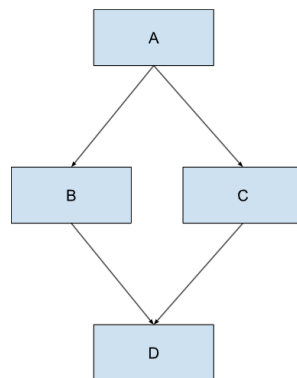


Figure 1: Il nodo A domina i nodi B, C e D perché per raggiungerli si deve per forza passare per il nodo A indipendentemente dal percorso scelto dal programma; mentre il nodo B NON domina il nodo D perché non è garantito che il programma passerà per il nodo B per arrivare al nodo D (stesso discorso viene applicato per il nodo C)

Framework

- **Dominio**

Insieme di nodi del CFG: N ; perché ogni iterazione lavora su sottoinsiemi di questi nodi, ogni $IN[B]$ e $OUT[B]$ è un sottoinsieme dell'insieme N .

- **Inizializzazione**

Annotiamo ogni basic block B_i con un insieme $DOM[B_i]$.

$B_i \in DOM[B_j] \Leftrightarrow B_i$ domina B_j .

Per definizione, ogni nodo domina sè stesso: $B_i \in DOM[B_i]$.

All'inizio, ogni elemento dell'insieme dei nodi del CFG appartiene a $DOM[B_i]$, dato che ogni nodo potrebbe dominare un'altro, tutti tranne il nodo ENTRY, il cui avrà come dominante solo se stesso (è il primo nodo, tutti i percorsi partono da lui).

- boundary condition:

$out[Entry] = \{Entry\}$

- initial interior points:

$out[b] = \{N\}$

- **Direzione**

Scegliamo di iniziare a percorrere il CFG dal nodo ENTRY fino al nodo EXIT, dall'altro verso il basso, imitando il flusso del programma.

Ogni nodo riceverà in ingresso il sottoinsieme ottenuto dall'intersezione dei nodi che dominano tutti i suoi nodi precedenti, in questo modo rimarranno solo i nodi che compaiono in TUTTI i cammini verso B).

E restituirà in uscita il sottoinsieme ottenuto dall'unione tra il sottoinsieme passato in ingresso e il nodo attuale (ogni nodo domina sè stesso).

- forward:

$out[b] = f_b(in(b))$

$in[b] = \bigwedge out[pred(b)]$

- meet operation:

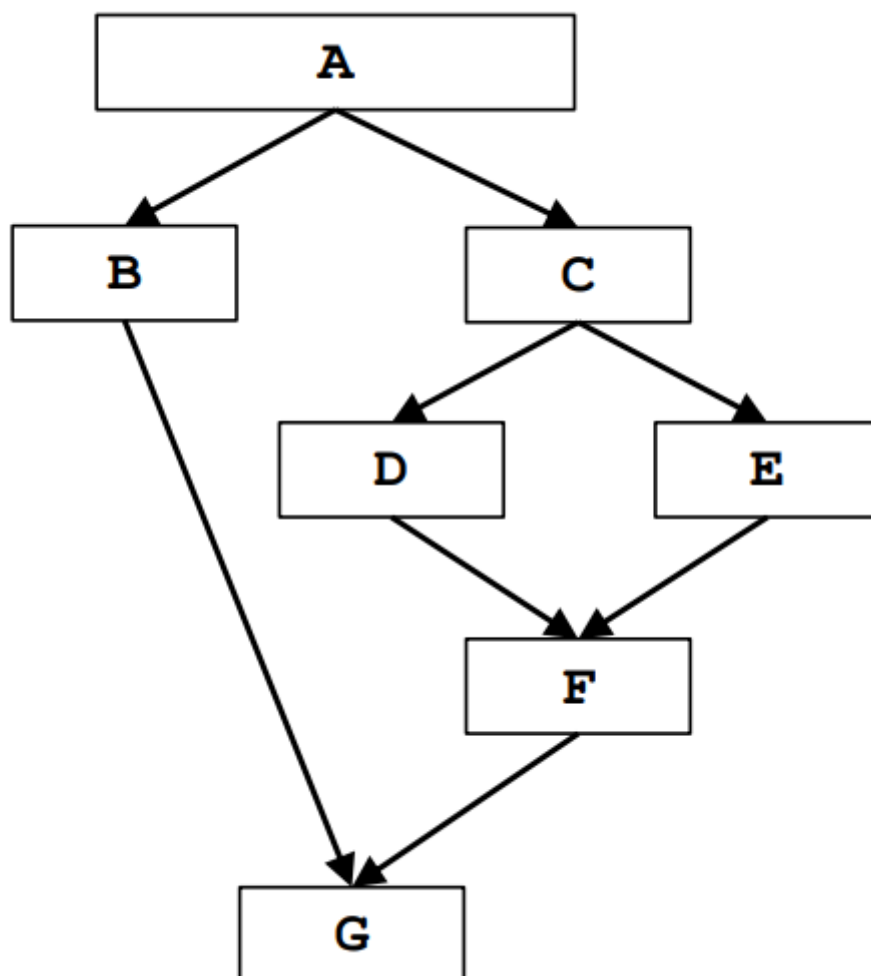
$\cap$

- Funzione di trasferimento

$f_b(x) = Dom_b \cup x$

	Dominator Analysis
Domain	Sets of Nodes
Direction	forward: $out[b] = f_b(in(b))$ $in[b] = \bigwedge out[pred(b)]$
Transfer function	$f_b(x) = Dom_b \cup x$
Meet Operation	$\cap$
Boundary Condition	$out[Entry] = \{Entry\}$
Initial interior points	$out[b] = \{N\}$

- **Tabella delle iterazioni**



	Iterazione 0 (Inizializzazione)	
	IN[B]	OUT[B]
A	$\{\emptyset\}$	$\{A\}$
B	$\{\emptyset\}$	$\{A,B,C,D,E,F,G\}$
C	$\{\emptyset\}$	$\{A,B,C,D,E,F,G\}$
D	$\{\emptyset\}$	$\{A,B,C,D,E,F,G\}$
E	$\{\emptyset\}$	$\{A,B,C,D,E,F,G\}$
F	$\{\emptyset\}$	$\{A,B,C,D,E,F,G\}$
G	$\{\emptyset\}$	$\{A,B,C,D,E,F,G\}$

OUT è uguale a tutto perché all'inizio assumiamo che tutti i nodi dominano tutti gli altri nodi
 IN è uguale all'insieme vuoto perché non abbiamo ancora propagato informazioni dai predecessori

	Iterazione 1	
	IN[B]	OUT[B]
A	$\{A\}$	$\{A\}$
B	$\{A\}$	$\{A,B\}$
C	$\{A\}$	$\{A,C\}$
D	$\{A,C\}$	$\{A,C,D\}$
E	$\{A,C\}$	$\{A,C,E\}$
F	$\{A,C\}$	$\{A,C,F\}$
G	$\{A\}$	$\{A,G\}$